

h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Seminar paper in the field of Mathematics and Natural Sciences
Bachelor's program in Applied Mathematics

Mean-Preserving Bézier Interpolation of Hydrochemical Time-Series Data under Physical Constraints

Yana Osinchuk

May 20, 2026

The topic presented
Prof. Dr. Thomas Maerz

Second examiner
All present

Declaration

I affirm that I have independently authored this seminar document and have not used sources or aids other than those explicitly mentioned. Any directly quoted or paraphrased passages from other works are appropriately acknowledged.

Osinchuk Yana

Summary

This study examines Bézier-based interpolation methods for the approximation of hydrochemical and hydrological time-series data subject to simultaneous mathematical and physical constraints. The main objective is to construct an interpolatory framework capable of producing a smooth representation of the measured process while satisfying prescribed endpoint, integral, and shape conditions. In particular, on each interpolation interval, the constructed curve is required to pass through the initial and terminal data values, preserve the prescribed interval mean, incorporate derivative information when available, and remain within physically admissible bounds.

Conventional interpolation schemes can generate smooth approximating curves; however, they do not, in general, guarantee the preservation of integral characteristics such as interval mean values. Moreover, standard interpolation methods may produce overshoots or undershoots that are inconsistent with the physical nature of hydrological and hydrochemical variables. To overcome these limitations, several modified Bézier formulations are developed and compared. The considered methods include a mean-preserving cubic Bézier construction, an exact quartic Bézier formulation with derivative constraints, and a modified bounded quartic Bézier formulation with soft derivative constraints. In the latter method, derivative estimates are treated as reference slopes rather than as unconditional constraints. The algorithms were first implemented and tested in Python, where numerical experiments and graphical diagnostics were performed, and were subsequently translated into Excel through Visual Basic for Applications (VBA) to enable their use within a spreadsheet-based operational workflow.

The performance of the proposed methods is assessed through visual comparison with the measured data and quantitative error analysis. The evaluation is based on standard statistical metrics, including the root-mean-square error RMSE, the mean absolute error MAE, the maximum absolute error, and the coefficient of determination R^2. The comparison is designed to assess not only approximation accuracy, but also the preservation of endpoint values, interval means, derivative behavior, and physical bounds.

Zusammenfassung

Diese Arbeit untersucht Bézier-basierte Interpolationsmethoden zur Approximation hydrochemischer und hydrologischer Zeitreihendaten unter gleichzeitiger Berücksichtigung mathematischer und physikalischer Nebenbedingungen. Das Hauptziel besteht darin, ein Interpolationsverfahren zu konstruieren, das eine glatte Darstellung des gemessenen Prozesses erzeugt und dabei vorgegebene Endpunkt-, Integral- und Formbedingungen erfüllt. Insbesondere soll die konstruierte Kurve auf jedem Interpolationsintervall durch die Anfangs- und Endwerte verlaufen, den vorgeschriebenen Intervallmittelwert erhalten, vorhandene Ableitungsinformationen einbeziehen und innerhalb physikalisch zulässiger Grenzen bleiben.

Konventionelle Interpolationsverfahren können glatte approximierende Kurven erzeugen; im Allgemeinen garantieren sie jedoch nicht die Erhaltung integraler Eigenschaften, wie etwa von Intervallmittelwerten. Darüber hinaus können Standardverfahren Über- oder Unterschwingungen erzeugen, die mit der physikalischen Bedeutung hydrologischer und hydrochemischer Größen nicht vereinbar sind. Um diese Einschränkungen zu überwinden, werden mehrere modifizierte Bézier-Formulierungen entwickelt und verglichen. Die betrachteten Methoden umfassen eine mittelwerterhaltende kubische Bézier-Konstruktion, eine exakte quartische Bézier-Formulierung mit Ableitungsbedingungen sowie eine modifizierte beschränkte quartische Bézier-Formulierung mit weichen Ableitungsbedingungen. Bei der letztgenannten Methode werden die Ableitungsschätzungen als Referenzsteigungen und nicht als uneingeschränkte Nebenbedingungen behandelt. Die Algorithmen wurden zunächst in Python implementiert und getestet, wobei numerische Experimente und graphische Diagnosen durchgeführt wurden. Anschließend wurden sie in Excel mittels Visual Basic for Applications (VBA) übertragen, um ihre Anwendung in einem tabellenbasierten Arbeitsablauf zu ermöglichen.

Die Leistungsfähigkeit der vorgeschlagenen Methoden wird durch einen visuellen Vergleich mit den gemessenen Daten sowie durch eine quantitative Fehleranalyse bewertet. Die Auswertung basiert auf Standardmetriken, darunter die Wurzel des mittleren quadratischen Fehlers RMSE, der mittlere absolute Fehler MAE, der maximale absolute Fehler und das Bestimmtheitsmaß R^2. Der Vergleich ist darauf ausgelegt, nicht nur die Approximationsgenauigkeit, sondern auch die Erhaltung der Endpunktwerte, der Intervallmittelwerte, das Verhalten der Ableitungen und die Einhaltung physikalischer Grenzen zu beurteilen. Insbesondere wird erwartet, dass die beschränkte quartische Bézier-Formulierung die physikalisch konsistenteste Darstellung liefert, da

sie Mittelwerterhaltung mit ableitungsbasierter Formkontrolle und expliziten Zulässigkeitsprüfungen verbindet.

Preface

This research was completed under the supervision of Prof. Dr. Thomas Maerz, to whom I would like to express my sincere and profound gratitude. His continuous support, thoughtful guidance, and steady encouragement were invaluable throughout the development of this work.

I am especially grateful for his willingness to discuss every difficulty in detail, for his clear and insightful explanations, and for the constructive feedback that accompanied this project from its initial ideas to its final implementation. His guidance not only strengthened my understanding of the interpolation methods investigated in this work, but also helped me translate the theoretical concepts into practical computational tools using Python and Excel.

Sincere appreciation is also expressed to Mr. Andreas Burmeister from the Hessian Agency for Nature Conservation, Environment and Geology. His practical expertise, valuable suggestions, and experience with real environmental data added an important applied dimension to this work. Specifically, his openness in sharing his professional perspective was instrumental in relating the mathematical methods studied herein to concrete questions arising in hydrological and hydrochemical data analysis.

Contents

Declaration i

Summary ii

Zusammenfassung iii

Preface v

1 Introduction 1

2 Theoretical Framework 3

2.1 Discrete Time-Series Data and Support Points 3

2.2 Interpolation and Approximation 4

2.3 Normalized Local Parameter 4

2.4 Interval Mean Constraint 5

2.5 Physical Bounds 6

2.6 Bernstein Basis Polynomials 6

2.7 Bézier Curves 7

2.8 Derivative Estimation 8

2.9 Error Metrics 9

2.10 Roughness of the Control Polygon 10

2.11 Piecewise Continuity 11

2.12 Summary of Required Constraints 11

3 Mean-Preserving Cubic Bézier Interpolation 12

4 Quartic Bézier Interpolation with Derivative and Bound Con-

straints **14**

4.1 Exact Quartic Bézier Formulation 14

4.2 Bounded Quartic Bézier Formulation with Soft Derivative Constraints 15

4.3 Feasibility of Physical Bounds 16

5 Implementation in Python and Excel/VBA **18**

5.1 Overall Computational Workflow 18

5.2 Data Preparation and Support Point Selection 19

5.3 Computation of Empirical Interval Means 20

5.4 Construction of the Mean-Preserving Cubic Bézier Method . . 21

5.5 Construction of the Exact Quartic Bézier Method 23

5.6 Derivative Estimation 24

5.7 Construction of the Bounded Quartic Bézier Method 25

5.8 Evaluation of the Piecewise Bézier Curves 28

5.9 Verification of Constraints 29

5.10 Error Analysis and Numerical Comparison 31

5.11 Control-Polygon Roughness 32

5.12 Graphical Diagnostics 33

5.13 Excel/VBA Implementation 33

5.14 Summary of the Implementation Logic 38

5.15 Source Codes 39

6 Numerical Results and Comparison **41**

6.1 Numerical Setup 41

6.2 Approximation Quality 42

6.3 Constraint Verification 43
6.4 Derivative Information 44
6.5 Graphical Comparison 45
6.6 Overall Interpretation 47

7 Conclusion **48**

References **50**

List of Figures **52**

List of Tables **53**

List of Code Listings **54**

1 Introduction

Hydrochemical and hydrological monitoring frequently results in limited time series data, as the laboratory analysis of chemical elements and water quality parameters can often be expensive, time-consuming, or experimentally challenging. In environmental applications, sparse observations are commonly combined with interpolation or approximation procedures in order to support further analysis, modelling, visualization, and decision making [1, 2, 3]. The available observations therefore describe the underlying process only at discrete sampling times, whereas many subsequent tasks, including visualization, comparison of data sets, numerical integration, gap filling, trend detection, and model coupling, require a representation on a finer temporal scale [1, 2].

The choice of an interpolation or model-construction method is not merely a technical detail. In environmental and geospatial modelling, different interpolation methods may lead to different derived quantities and therefore affect subsequent calculations. For example, studies on digital elevation models and soil erosion show that the method used to construct an intermediate numerical surface can influence the results of later environmental computations [4]. This motivates the use of interpolation methods that are not only accurate, but also consistent with physical and statistical properties of the measured data.

In this paper, interpolation is treated as a constrained reconstruction problem rather than as a purely graphical smoothing procedure. On each interval, the interpolant should reproduce the measured endpoint values, preserve the prescribed interval mean, respect physically admissible bounds, and incorporate derivative information only when it is compatible with these restrictions. This viewpoint is related to classical interpolation and approximation of tabular data [5, 6], as well as to shape-preserving polynomial interpolation, where slopes or control parameters are restricted in order to avoid artificial extrema, loss of monotonicity, or violation of positivity [7, 8, 9].

The chosen framework is based on Bézier polynomials. Bézier curves provide local geometric control through their control points: they interpolate the first and last control points, allow endpoint derivative information to be represented algebraically, and remain within the convex hull of the control polygon [10, 11, 12, 13]. These properties are particularly useful for hydrochemical data, since a visually smooth curve may still be physically inappropriate if it overshoots feasible concentration ranges or changes the average value represented by the measurements [3, 4]. Bézier curves and Bernstein polynomials are classical

tools in computer-aided geometric design and curve representation [10, 11, 14].

Three Bézier-based constructions are investigated. First, a mean-preserving cubic segment is derived by selecting the inner control points so that the interval average is conserved. Second, an exact quartic Bézier formulation is used as a reference and diagnostic model, in which endpoint derivatives and interval means are imposed simultaneously. Third, a bounded quartic formulation is proposed, in which derivative estimates are treated as reference slopes: they are retained if they are compatible with the physical bounds and otherwise modified through projection onto the feasible set. The distinction between hard and soft derivative constraints follows the general idea of shape-preserving interpolation, where local derivative information is useful but must be controlled when it conflicts with admissibility [7, 8].

The algorithms were first developed in Python for numerical experimentation, derivative estimation, graphical diagnostics, and error analysis. The final procedures were then implemented in Excel/VBA to make the methods usable in a spreadsheet-based workflow for hydrological and hydrochemical data. The paper is organized as follows: Section 2 recalls the required background on interpolation, Bézier curves, mean preservation, physical bounds, derivative estimation, and error metrics. Section 3 develops the mean-preserving cubic construction. Section 4 presents the quartic formulations, including the constrained method with soft derivative conditions. Section 5 describes the Python and Excel/VBA implementations. Section 6 discusses the numerical results and comparison of the methods. Section 7 summarizes the conclusions and possible extensions.

2 Theoretical Framework

This section introduces the mathematical concepts and notation used throughout the paper. The aim is to define the interpolation problem, the prescribed constraints, the Bézier representation, and the quantitative criteria used to evaluate the resulting curves. The notation follows standard numerical analysis and geometric modelling literature on interpolation, piecewise polynomial curves, Bernstein polynomials, and Bézier representations [5, 6, 10, 11].

2.1 Discrete Time-Series Data and Support Points

Let the measured data be given as a discrete time series

$$(x_0^{\mathrm{ref}}, y_0^{\mathrm{ref}}), (x_1^{\mathrm{ref}}, y_1^{\mathrm{ref}}), \ldots, (x_N^{\mathrm{ref}}, y_N^{\mathrm{ref}}),$$

where x_j^{ref} denotes time and y_j^{ref} denotes the measured value of the hydrological or hydrochemical quantity considered. Discrete tabular data of this form are the standard starting point for interpolation and approximation problems [5, 6]. It is assumed that the time values are strictly increasing,

$$x_0^{\mathrm{ref}} < x_1^{\mathrm{ref}} < \cdots < x_N^{\mathrm{ref}}.$$

In practical applications, the complete measured data set may contain many values, while the interpolation method is constructed only on a smaller set of support points. These support points are denoted by

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n),$$

where

$$x_0 < x_1 < \cdots < x_n.$$

The interval between two neighboring support points is written as

$$I_i = [x_i, x_{i+1}], \qquad i = 0, \ldots, n-1,$$

and its length is

$$h_i = x_{i+1} - x_i.$$

The use of support points is closely connected to local piecewise interpolation: instead of constructing one global high-degree polynomial, the global curve is assembled from local segments, which improves stability and local control [6, 10].

2.2 Interpolation and Approximation

Interpolation is the construction of a function F that passes through prescribed data points. In the present setting, the interpolation condition is

$$F(x_i) = y_i, \qquad i = 0, \ldots, n.$$

Since the curve is constructed locally on each interval I_i, the interpolant can be written as a piecewise function

$$F(x) = F_i(x), \qquad x \in [x_i, x_{i+1}].$$

Each local function F_i must satisfy the endpoint interpolation conditions

$$F_i(x_i) = y_i, \qquad F_i(x_{i+1}) = y_{i+1}.$$

Classical numerical analysis distinguishes interpolation, where selected data values are reproduced exactly, from approximation, where the constructed function is evaluated by how well it represents a larger data set [5, 6]. In this work both ideas are present. The constructed function interpolates the selected support points exactly, but it is also evaluated as an approximation of the full measured reference data. Similar interpolation and approximation ideas are frequently used in water-quality and environmental time-series analysis when data are available only at discrete measurement times [1, 2].

2.3 Normalized Local Parameter

For the construction of local Bézier curves, each interval $I_i = [x_i, x_{i+1}]$ is mapped to the standard interval $[0, 1]$. This normalization is standard in Bézier and spline curve representations, where polynomial segments are usually expressed in a local parameter rather than directly in the physical coordinate [10, 11]. The normalized local parameter is defined by

$$t = \frac{x - x_i}{h_i}, \qquad h_i = x_{i+1} - x_i.$$

Thus,

$$x = x_i + h_i t, \qquad t \in [0, 1].$$

This transformation allows every local segment to be treated on the same parameter interval independently of its physical length. It also simplifies the formulas for Bézier control points, mean values, and endpoint derivatives [10, 12].

2.4 Interval Mean Constraint

For each interval between two neighboring support points, a prescribed mean value is computed from the original reference measurements. Let s_i and e_i denote the indices of the first and last reference data points belonging to the interval $I_i = [x_i, x_{i+1}]$. The prescribed empirical interval mean is defined by

$$m_i = \frac{1}{N_i} \sum_{j=s_i}^{e_i} y_j^{\mathrm{ref}}, \qquad N_i = e_i - s_i + 1.$$

Thus, m_i is the arithmetic average of all measured reference values inside the interval, including both endpoints. This corresponds to the implementation in which all values between two consecutive support indices are summed and divided by the number of included values. Arithmetic averages of measured tabular values are commonly used as simple empirical representatives of interval behavior, especially when the data are sampled at a regular time step [5, 1].

The interpolating curve is then required to preserve this empirical mean. For a local interpolating function F_i on the interval $I_i = [x_i, x_{i+1}]$, the continuous mean value over this interval is defined by

$$\frac{1}{h_i} \int_{x_i}^{x_{i+1}} F_i(x)\, dx, \qquad h_i = x_{i+1} - x_i.$$

Using the normalized local parameter, this expression can be rewritten as

$$\frac{1}{h_i} \int_{x_i}^{x_{i+1}} F_i(x)\, dx = \int_0^1 F_i(x_i + h_i t)\, dt.$$

The mean-preserving condition therefore requires

$$\int_0^1 F_i(x_i + h_i t)\, dt = m_i.$$

In other words, the prescribed mean m_i is computed as an arithmetic average of discrete measured data, whereas the interpolating curve is required to have the same continuous average over the corresponding interval. This condition ensures that the interpolation not only matches endpoint values, but also preserves an integral characteristic of the original data. The importance of preserving physically meaningful averages is consistent with environmental modelling practice, where interpolation may influence later computations and interpretations [3, 4].

2.5 Physical Bounds

In hydrochemical and environmental applications, physical admissibility is as important as numerical smoothness, because interpolation may otherwise create values that have no meaningful interpretation. For example, water levels or concentrations should not become negative. More generally, one may require the interpolating curve to remain inside a physically admissible interval

$$L \leq F_i(x) \leq U, \qquad x \in I_i,$$

where L is a lower bound and U is an optional upper bound [3, 1]. If no upper bound is prescribed, one may formally set $U = +\infty$.

In this work, physical admissibility is enforced through the control points of the Bézier curve. This gives a simple sufficient condition for boundedness: if all control points of a scalar Bézier segment lie inside the interval $[L, U]$, then the complete Bézier curve also lies inside $[L, U]$. This follows from the convex hull property of Bézier curves, which is a standard property derived from the non-negativity and partition-of-unity properties of the Bernstein basis [10, 11, 13, 12].

2.6 Bernstein Basis Polynomials

The Bernstein basis polynomials of degree d are defined by

$$b_{k,d}(t) = \binom{d}{k}(1-t)^{d-k}t^k, \qquad k = 0, \ldots, d, \qquad t \in [0, 1].$$

The Bernstein basis is the classical polynomial basis used in Bézier curve representations [10, 11, 14]. The basis functions are non-negative on $[0, 1]$,

$$b_{k,d}(t) \geq 0,$$

and form a partition of unity,

$$\sum_{k=0}^{d} b_{k,d}(t) = 1.$$

These two properties imply that any Bézier curve is a convex combination of its control points for every $t \in [0, 1]$. This is the mathematical basis of the convex hull property used later to enforce physical bounds [13, 12].

2.7 Bézier Curves

A scalar Bézier curve of degree d with control points

$$P_0, P_1, \ldots, P_d$$

is defined as

$$B(t) = \sum_{k=0}^{d} P_k b_{k,d}(t), \qquad t \in [0,1].$$

Equivalently,

$$B(t) = \sum_{k=0}^{d} P_k \binom{d}{k} (1-t)^{d-k} t^k.$$

This representation is standard in computer-aided geometric design and is valued because it combines polynomial simplicity with geometric control through the control polygon [10, 11]. The first and last control points are exactly interpolated:

$$B(0) = P_0, \qquad B(1) = P_d.$$

Therefore, by choosing

$$P_0 = y_i, \qquad P_d = y_{i+1},$$

one obtains a Bézier segment that passes through the endpoints of the interval [12, 10].

The Bézier curve lies within the convex hull of its control points. In the scalar case, this means

$$\min_{0 \le k \le d} P_k \le B(t) \le \max_{0 \le k \le d} P_k, \qquad t \in [0,1].$$

Consequently, if

$$L \le P_k \le U \qquad \text{for all } k,$$

then

$$L \le B(t) \le U \qquad \text{for all } t \in [0,1].$$

This property is one of the main reasons why Bézier curves are suitable for physically constrained interpolation [10, 11, 13].

A further property that is essential for this work is the integral of a Bézier curve over the normalized interval. Since

$$\int_0^1 b_{k,d}(t)\, dt = \frac{1}{d+1},$$

it follows that

$$\int_0^1 B(t)\,dt = \frac{1}{d+1}\sum_{k=0}^{d} P_k.$$

Therefore, the mean value of a Bézier curve over $[0,1]$ is equal to the arithmetic mean of its control points. This simple integral formula is a direct consequence of the Bernstein basis and is central to the mean-preserving construction used in this work [10, 14].

The derivative of a Bézier curve is again a Bézier curve of one degree lower. In particular,

$$\frac{dB}{dt}(t) = d\sum_{k=0}^{d-1}(P_{k+1} - P_k)b_{k,d-1}(t).$$

Thus, the endpoint derivatives with respect to the local parameter t are

$$\frac{dB}{dt}(0) = d(P_1 - P_0), \qquad \frac{dB}{dt}(1) = d(P_d - P_{d-1}).$$

Because $x = x_i + h_i t$, derivatives with respect to the original variable x satisfy

$$\frac{dB}{dx} = \frac{1}{h_i}\frac{dB}{dt}.$$

These derivative formulas allow endpoint derivative estimates to be translated directly into inner control points [10, 11, 12].

2.8 Derivative Estimation

In most practical data sets, derivative values are not measured directly. Therefore, they must be estimated from neighboring support points. The simplest estimate is based on secant slopes. On the interval $[x_i, x_{i+1}]$, the secant slope is

$$\delta_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i} = \frac{y_{i+1} - y_i}{h_i}.$$

A basic finite-difference approximation for an interior derivative is

$$d_i \approx \frac{y_{i+1} - y_{i-1}}{x_{i+1} - x_{i-1}}.$$

Finite differences are simple and widely used in numerical computation, but for irregular or noisy data they may produce large derivative values and may lead to overshoot. For this reason, shape-preserving derivative estimates are often preferable in interpolation problems [6, 7, 8].

In this work, a PCHIP-like derivative estimate is used. For an interior support point x_i, let δ_{i-1} and δ_i be the neighboring secant slopes. If these slopes have opposite signs, or if one of them is zero, the derivative is set to zero:

$$d_i = 0 \qquad \text{if} \qquad \delta_{i-1}\delta_i \leq 0.$$

This prevents the introduction of artificial local extrema. If the two neighboring secant slopes have the same sign, the derivative is defined as a weighted harmonic mean,

$$d_i = \frac{w_1 + w_2}{\dfrac{w_1}{\delta_{i-1}} + \dfrac{w_2}{\delta_i}},$$

where

$$w_1 = 2h_i + h_{i-1}, \qquad w_2 = h_i + 2h_{i-1}.$$

This construction follows the idea of monotone piecewise cubic Hermite interpolation, where derivative values are selected locally to reduce overshoot and preserve the qualitative shape of the data [7, 8]. The SciPy implementation of `PchipInterpolator` is based on the same principle and documents its monotonicity-preserving and non-overshooting behavior for non-smooth data [9].

2.9 Error Metrics

To evaluate the quality of the interpolation, the constructed curve is compared with the measured reference data. Let

$$\widehat{y}_j = F(x_j^{\mathrm{ref}})$$

be the interpolated value at the reference point x_j^{ref}. The residual is defined as

$$e_j = y_j^{\mathrm{ref}} - \widehat{y}_j.$$

Standard model-evaluation practice often uses several error measures simultaneously, because different metrics emphasize different properties of the residuals [15, 16]. The root-mean-square error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{N+1}\sum_{j=0}^{N} e_j^2}.$$

The mean absolute error is

$$\mathrm{MAE} = \frac{1}{N+1}\sum_{j=0}^{N} |e_j|.$$

The maximum absolute error is

$$E_{\max} = \max_{0 \le j \le N} |e_j|.$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{j=0}^{N} \left(y_j^{\text{ref}} - \widehat{y}_j \right)^2}{\sum_{j=0}^{N} \left(y_j^{\text{ref}} - \bar{y}^{\text{ref}} \right)^2},$$

where

$$\bar{y}^{\text{ref}} = \frac{1}{N+1} \sum_{j=0}^{N} y_j^{\text{ref}}.$$

A smaller value of RMSE, MAE, or $E_{\max}$ indicates a better approximation. A value of R^2 close to 1 indicates that the interpolating curve explains most of the variation in the reference data. RMSE and MAE are both useful because RMSE penalizes large deviations more strongly, while MAE gives a more direct average absolute error [15, 16].

2.10 Roughness of the Control Polygon

In addition to error metrics, it is useful to consider a simple measure of local roughness. Smoothness and local bending are classical concerns in spline and geometric curve design [6, 10]. For a Bézier segment with control points $P_0, \ldots, P_d$, a discrete roughness indicator can be defined using second differences:

$$\mathcal{R}_i = \sum_{k=0}^{d-2} \left(P_k - 2P_{k+1} + P_{k+2} \right)^2.$$

For the complete piecewise interpolant, the total roughness is

$$\mathcal{R} = \sum_{i=0}^{n-1} \mathcal{R}_i.$$

This quantity is not an interpolation constraint. It is used only as a diagnostic indicator: large values of $\mathcal{R}$ may indicate strong local bending or possible oscillatory behavior. The idea is consistent with the general use of derivative or curvature-related quantities to evaluate the regularity of piecewise polynomial curves [6, 10].

2.11 Piecewise Continuity

The global interpolant is constructed by joining local Bézier segments. Since the right endpoint of one segment and the left endpoint of the next segment are both equal to the same support value, the piecewise curve is continuous:

$$F_i(x_{i+1}) = y_{i+1} = F_{i+1}(x_{i+1}).$$

Thus, $F \in C^0$. Piecewise polynomial interpolation often distinguishes between continuity of function values and continuity of derivatives; this distinction is fundamental in spline theory and curve design [6, 10].

Higher-order smoothness, such as C^1-continuity, requires the derivatives of neighboring segments to match at the shared support point. If the same derivative estimate is used from both sides, derivative continuity can be encouraged. However, in the bounded method, derivative values may be adjusted in order to preserve physical admissibility. Therefore, exact C^1-continuity is not always imposed as a hard requirement. This trade-off between smoothness and shape preservation is consistent with the philosophy of monotone and shape-preserving interpolation [7, 8].

2.12 Summary of Required Constraints

The interpolation methods considered in this work are designed to satisfy several conditions simultaneously. For each interval $I_i = [x_i, x_{i+1}]$, the desired requirements are

$$F_i(x_i) = y_i, \qquad F_i(x_{i+1}) = y_{i+1},$$
$$\int_0^1 F_i(x_i + h_i t)\, dt = m_i,$$

and

$$L \le F_i(x) \le U, \qquad x \in I_i.$$

When derivative information is used, the reference derivative conditions are

$$F_i'(x_i) \approx d_i, \qquad F_i'(x_{i+1}) \approx d_{i+1}.$$

In the exact quartic formulation, these derivative conditions are imposed as hard constraints. In the bounded quartic formulation, they are treated as soft reference conditions and are adjusted if necessary to preserve the physical bounds. This distinction combines ideas from Bézier curve control, convex-hull-based admissibility, and shape-preserving derivative estimation [10, 11, 7, 8].

3 Mean-Preserving Cubic Bézier Interpolation

A cubic Bézier curve has degree 3 and four control points

$$P_0, P_1, P_2, P_3.$$

It is given by

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3.$$

The cubic Bézier curve is one of the most commonly used polynomial Bézier forms because it provides a useful balance between simplicity and shape control [10, 11, 14]. The endpoint interpolation conditions are

$$B(0) = P_0, \qquad B(1) = P_3.$$

For the interval $I_i = [x_i, x_{i+1}]$, we set

$$P_0 = y_i, \qquad P_3 = y_{i+1}.$$

The mean value of the cubic Bézier segment is

$$\int_0^1 B(t)\, dt = \frac{P_0 + P_1 + P_2 + P_3}{4},$$

which follows directly from the integral property of the Bernstein basis [10, 13]. Therefore, the mean-preserving condition is

$$\frac{P_0 + P_1 + P_2 + P_3}{4} = m_i.$$

Since P_0 and P_3 are fixed by the endpoint values, the two inner control points must satisfy

$$P_1 + P_2 = 4m_i - P_0 - P_3.$$

This shows that the cubic Bézier formulation has one remaining degree of freedom after the endpoint and mean conditions have been imposed. The situation is typical for constrained interpolation: after the required interpolation and mean constraints are imposed, an additional selection principle is needed to obtain a unique segment [5, 6].

To obtain a unique unconstrained cubic segment, the remaining degree of freedom can be fixed by choosing the smoothest segment in the sense of the

second derivative. More precisely, the inner control points may be chosen to minimize

$$J(P_1, P_2) = \int_0^1 \left(B''(t)\right)^2 \, dt$$

subject to the linear constraint

$$P_1 + P_2 = 4m_i - P_0 - P_3.$$

Second-derivative or curvature-related criteria are commonly used as smoothness indicators in spline and curve design [6, 10]. This yields the preferred inner control points

$$P_1^{\text{pref}} = 2m_i - \frac{1}{3}P_0 - \frac{2}{3}P_3, \qquad P_2^{\text{pref}} = 2m_i - \frac{2}{3}P_0 - \frac{1}{3}P_3.$$

These formulas preserve constant data exactly: if $P_0 = P_3 = m_i$, then $P_1^{\text{pref}} = P_2^{\text{pref}} = m_i$.

If physical bounds are prescribed, the preferred pair $(P_1^{\text{pref}}, P_2^{\text{pref}})$ is projected onto the feasible set determined by

$$P_1 + P_2 = 4m_i - P_0 - P_3, \qquad L \le P_1, P_2 \le U.$$

This projection preserves the endpoint values and the interval mean exactly while ensuring that the Bézier segment remains inside the prescribed physical bounds whenever the feasible set is non-empty. The use of control-point bounds is justified by the convex hull property of Bézier curves [10, 11, 13].

4 Quartic Bézier Interpolation with Derivative and Bound Constraints

4.1 Exact Quartic Bézier Formulation

A quartic Bézier curve has degree 4 and five control points

$$P_0, P_1, P_2, P_3, P_4.$$

It is defined by

$$B(t) = (1-t)^4 P_0 + 4(1-t)^3 t P_1 + 6(1-t)^2 t^2 P_2 + 4(1-t) t^3 P_3 + t^4 P_4.$$

Higher-degree Bézier curves provide additional degrees of freedom compared with cubic curves, but still preserve the same Bernstein basis structure and geometric interpretation [10, 11]. The endpoint interpolation conditions are

$$B(0) = P_0, \qquad B(1) = P_4.$$

For the interval $I_i = [x_i, x_{i+1}]$, we choose

$$P_0 = y_i, \qquad P_4 = y_{i+1}.$$

The mean value of the quartic Bézier segment is

$$\int_0^1 B(t)\, dt = \frac{P_0 + P_1 + P_2 + P_3 + P_4}{5},$$

again using the Bernstein integral formula [10, 13]. Thus, the mean-preserving condition is

$$\frac{P_0 + P_1 + P_2 + P_3 + P_4}{5} = m_i,$$

or equivalently,

$$P_1 + P_2 + P_3 = 5m_i - P_0 - P_4.$$

The endpoint derivatives with respect to x are

$$B'_x(x_i) = \frac{4(P_1 - P_0)}{h_i}, \qquad B'_x(x_{i+1}) = \frac{4(P_4 - P_3)}{h_i}.$$

These formulas follow from the derivative property of Bézier curves and the local parameter transformation $x = x_i + h_i t$ [10, 12]. If derivative values d_i and d_{i+1} are imposed exactly, then

$$P_1 = P_0 + \frac{h_i}{4} d_i, \qquad P_3 = P_4 - \frac{h_i}{4} d_{i+1}.$$

The remaining middle control point is then determined by the mean condition:

$$P_2 = 5m_i - P_0 - P_1 - P_3 - P_4.$$

Hence, a quartic Bézier curve has exactly enough degrees of freedom to satisfy five scalar conditions: two endpoint values, one interval mean, and two endpoint derivatives. However, if all these conditions are imposed exactly, the control points are uniquely determined. Consequently, physical bounds cannot always be satisfied simultaneously. This is analogous to other constrained interpolation settings, where imposing too many hard conditions may remove the degrees of freedom needed for shape preservation [7, 8]. For this reason, the exact quartic formulation is useful as a diagnostic reference, but it is not always physically admissible.

4.2 Bounded Quartic Bézier Formulation with Soft Derivative Constraints

In the bounded quartic method, the endpoint values and interval mean are kept as exact constraints, while the derivatives are treated as soft reference conditions. A derivative condition is called a hard constraint if it must be satisfied exactly. In the quartic Bézier case, the hard derivative constraints would be

$$\frac{4(P_1 - P_0)}{h_i} = d_i, \qquad \frac{4(P_4 - P_3)}{h_i} = d_{i+1}.$$

These equations uniquely determine P_1 and P_3. Together with endpoint interpolation and mean preservation, all five quartic control points are then fixed.

In contrast, a derivative condition is called a soft constraint if the derivative estimate is used only as a reference value. In this case, the derivative-based control points

$$P_1^{\text{ref}} = P_0 + \frac{h_i}{4} d_i, \qquad P_3^{\text{ref}} = P_4 - \frac{h_i}{4} d_{i+1}$$

are regarded as preferred values, but they may be adjusted if they would cause a violation of the physical bounds. This idea follows the same philosophy as shape-preserving interpolation: derivative information is useful for local shape control, but it must be limited when it creates nonphysical behavior or overshoot [7, 8, 9]. The resulting curve still preserves the endpoints and the interval mean exactly, but the endpoint derivatives are allowed to deviate from the reference slopes when necessary.

The bounded construction can be formulated as a projection problem. For the quartic case, define

$$S_i = 5m_i - P_0 - P_4.$$

The inner control points must satisfy

$$P_1 + P_2 + P_3 = S_i.$$

Since

$$P_2 = S_i - P_1 - P_3,$$

it is sufficient to determine P_1 and P_3. The feasible set is

$$\mathcal{C}_i = \{(P_1, P_3) : L \le P_1 \le U,\ L \le P_3 \le U,\ L \le S_i - P_1 - P_3 \le U\}.$$

The bounded quartic construction chooses P_1 and P_3 as close as possible to the derivative-based reference values. This can be written as

$$(P_1, P_3) = \underset{(q_1,q_3)\in\mathcal{C}_i}{\operatorname{argmin}} \left[(q_1 - P_1^{\mathrm{ref}})^2 + (q_3 - P_3^{\mathrm{ref}})^2\right].$$

After P_1 and P_3 have been determined, the middle control point is obtained from

$$P_2 = S_i - P_1 - P_3.$$

This projection preserves the endpoint values and the interval mean exactly while enforcing the prescribed physical bounds whenever the feasible set is non-empty. The validity of this bound-enforcement strategy relies on the convex hull property of Bézier curves [10, 11, 13].

4.3 Feasibility of Physical Bounds

The physical bounds and the exact mean condition may be incompatible. Therefore, it is important to distinguish between an algorithmic failure and a mathematically infeasible interval [6, 7].

For a quartic Bézier segment with lower bound L and upper bound U, the internal control points must satisfy

$$P_1 + P_2 + P_3 = 5m_i - P_0 - P_4.$$

If all control points are required to lie in $[L, U]$, then the sum of the three internal control points must satisfy

$$3L \le 5m_i - P_0 - P_4 \le 3U.$$

If no upper bound is prescribed, the upper inequality is omitted. In particular, if only the lower bound $L = 0$ is used, a necessary feasibility condition is

$$5m_i - P_0 - P_4 \geq 0,$$

or equivalently,

$$m_i \geq \frac{P_0 + P_4}{5}.$$

If this condition fails, no quartic Bézier segment with non-negative control points can satisfy both the endpoint values and the exact mean condition. This feasibility check is therefore a mathematical part of the method, not only an implementation detail. It ensures that physical admissibility is treated as an explicit constraint rather than as an after-the-fact graphical correction [10, 3].

5 Implementation in Python and Excel/VBA

The methods described in the previous sections were first implemented in Python. The Python implementation was used for the development and testing of the mathematical construction, for derivative estimation, evaluation of the piecewise Bézier curves, graphical comparison, error analysis, and verification of the imposed constraints. After the numerical behavior of the methods had been checked in Python, the final bounded quartic Bézier method was translated into Excel/VBA.

The implementation follows the same logical structure for all considered methods. First, the measured data are loaded and converted into a numerical time variable. Then a smaller set of support points is selected. For every interval between two neighboring support points, an empirical mean value is computed from the original measured data. These values are then used as exact mean constraints in the construction of the Bézier control points. Finally, the resulting piecewise curves are evaluated, verified, and compared with the original reference data.

5.1 Overall Computational Workflow

The complete Python workflow is organized in a comparison script. This script loads the reference data, constructs the support points, computes the interval means, applies the three Bézier-based interpolation methods, evaluates the curves on a dense grid, verifies the constraints, computes error metrics, and plots the results.

Code Listing 1: Main structure of the Python comparison workflow.

```
def main():
    data_path = ROOT / "data" / "pegelonline_leunneu_2024.xlsx"

    step = 672
    num_points = 500

    lower_bound = 0.0
    upper_bound = None
    derivative_method = "pchip"

    x_ref, y_ref = load_reference_data(data_path)
    idx, x_nodes, y_nodes = choose_support_points(x_ref, y_ref, step)
    means = compute_interval_means_from_reference(y_ref, idx)
```

```
results = {}

# 1) Cubic B\'ezier with exact means and bounds
# 2) Exact quartic B\'ezier with derivatives
# 3) Bounded quartic B\'ezier with soft derivatives

print_metrics_table(results)
print_constraint_report(results)

plot_comparison(x_ref, y_ref, x_nodes, y_nodes, results)
plot_residuals(results)

plt.show()
```

This structure is useful because all methods are tested under the same conditions. They use the same support points, the same empirical interval means, the same physical bounds, and the same comparison grid. Therefore, the resulting error metrics and constraint reports can be compared directly.

5.2 Data Preparation and Support Point Selection

The reference data are read from an Excel file containing two columns: timestamps and measured values.

timestamp	value
Mon/ 01/01/2024 00:00	78
Mon/ 01/01/2024 00:15	78
Mon/ 01/01/2024 00:30	78
Mon/ 01/01/2024 00:45	78
Mon/ 01/01/2024 01:00	78
Mon/ 01/01/2024 01:15	78
Mon/ 01/01/2024 01:30	78
Mon/ 01/01/2024 01:45	78
Mon/ 01/01/2024 02:00	78
Mon/ 01/01/2024 02:15	77
Mon/ 01/01/2024 02:30	77

Figure 1: Example of a table

Since Bézier interpolation is performed on numerical variables, the timestamp column is first converted into a numerical time scale. In the implementation, the first timestamp is used as the origin, and all later timestamps are expressed as hours since this initial time.

Code Listing 2: Loading and preprocessing of the reference data.

```
def load_reference_data(excel_path):
    df = pd.read_excel(excel_path)

    df = df[["timestamp", "value"]].dropna().copy()
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.sort_values("timestamp").drop_duplicates(subset="timestamp")

    t0 = df["timestamp"].iloc[0]
    df["x_hours"] = (df["timestamp"] - t0).dt.total_seconds() / 3600.0

    return df["x_hours"].to_numpy(dtype=float), df["value"].to_numpy(dtype
  ↪ =float)
```

After the reference data have been loaded, support points are selected at a prescribed step size. In the numerical experiments, the value `step` $= 672$ was used, representing a week as a distance. The final data point is added explicitly if it is not already included, so that the interpolation always covers the full observation interval.

Code Listing 3: Selection of support points from the reference data.

```
def choose_support_points(x_ref, y_ref, step):
    idx = np.arange(0, len(x_ref), step)

    if idx[-1] != len(x_ref) - 1:
        idx = np.append(idx, len(x_ref) - 1)

    return idx, x_ref[idx], y_ref[idx]
```

Thus, the full reference data set is used for evaluation and comparison, while the Bézier interpolants themselves are constructed only from the selected support values.

5.3 Computation of Empirical Interval Means

For each interval between two consecutive support points, the prescribed mean value is computed from the original reference data. Let a and b be the indices of two neighboring support points. Then all measured values

$$y_a^{\mathrm{ref}}, y_{a+1}^{\mathrm{ref}}, \ldots, y_b^{\mathrm{ref}}$$

are included in the empirical mean. In Python, the expression `y_ref[a:b+1]` is used because the upper bound of a Python slice is not included.

Code Listing 4: Computation of empirical interval means from reference data.

```
def compute_interval_means_from_reference(y_ref, idx):
    means = []

    for i in range(len(idx) - 1):
        a = idx[i]
        b = idx[i + 1]
        means.append(np.mean(y_ref[a:b + 1]))

    return np.asarray(means, dtype=float)
```

Mathematically, this corresponds to

$$m_i = \frac{1}{b - a + 1} \sum_{j=a}^{b} y_j^{\mathrm{ref}}.$$

5.4 Construction of the Mean-Preserving Cubic Bézier Method

The cubic method constructs one cubic Bézier segment on each interval. In the implementation, preferred values for P_1 and P_2 are first computed. If physical bounds are prescribed, this preferred pair is then projected onto the feasible interval defined by the exact mean constraint and the lower and upper bounds.

Code Listing 5: Construction of cubic Bézier control points with exact mean preservation.

```
def _compute_bezier_control_points(
    y_nodes,
    means,
    lower_bound=None,
    upper_bound=None,
):
    n_intervals = len(y_nodes) - 1
    controls = np.zeros((n_intervals, 4), dtype=float)

    for i in range(n_intervals):
        y0 = y_nodes[i]
        y1 = y_nodes[i + 1]
```

```
        m = means[i]

        P0 = y0
        P3 = y1

        preferred_P1 = 2.0 * m - (1.0 / 3.0) * y0 - (2.0 / 3.0) * y1
        preferred_P2 = 2.0 * m - (2.0 / 3.0) * y0 - (1.0 / 3.0) * y1

        P1, P2 = _enforce_bounded_mean_pair(
            preferred_P1,
            preferred_P2,
            target_sum=4.0 * m - y0 - y1,
            lower_bound=lower_bound,
            upper_bound=upper_bound,
            interval_index=i,
        )

        controls[i] = [P0, P1, P2, P3]

    return controls
```

The auxiliary projection function keeps the exact mean condition unchanged. The value of P_1 is restricted to the feasible interval, and P_2 is then determined from the required sum.

Code Listing 6: Projection of the cubic internal control points onto the bounded mean-preserving feasible set.

```
def _enforce_bounded_mean_pair(
    preferred_p1,
    preferred_p2,
    target_sum,
    lower_bound=None,
    upper_bound=None,
    interval_index=None,
):
    if lower_bound is None and upper_bound is None:
        return float(preferred_p1), float(preferred_p2)

    lower = -np.inf if lower_bound is None else float(lower_bound)
    upper = np.inf if upper_bound is None else float(upper_bound)

    feasible_low = max(lower, target_sum - upper)
    feasible_high = min(upper, target_sum - lower)

    if feasible_low > feasible_high:
        raise ValueError(
            "No feasible bounded B\'ezier segment for interval "
            f"{interval_index}: exact mean preservation conflicts "
```

```
            f"with the requested bounds."
        )

    projected_p1 = 0.5 * (preferred_p1 - preferred_p2 + target_sum)
    p1 = min(max(projected_p1, feasible_low), feasible_high)
    p2 = target_sum - p1

    return float(p1), float(p2)
```

Because a scalar Bézier curve remains inside the convex hull of its control points, bounded control points are sufficient to guarantee that the whole cubic segment remains within the same physical bounds.

5.5 Construction of the Exact Quartic Bézier Method

Code Listing 7: Exact quartic Bézier construction with endpoint, mean, and derivative constraints.

```
def _compute_quartic_bezier_control_points(
    x_nodes,
    y_nodes,
    means,
    derivatives,
    lower_bound,
    upper_bound,
    strict_bounds,
):
    n_intervals = len(x_nodes) - 1
    controls = np.zeros((n_intervals, 5), dtype=float)

    for i in range(n_intervals):
        h = x_nodes[i + 1] - x_nodes[i]

        y0 = y_nodes[i]
        y1 = y_nodes[i + 1]
        m = means[i]

        d0 = derivatives[i]
        d1 = derivatives[i + 1]

        P0 = y0
        P1 = y0 + (h / 4.0) * d0
        P3 = y1 - (h / 4.0) * d1
        P4 = y1
        P2 = 5.0 * m - P0 - P1 - P3 - P4
```

```
        segment_controls = np.array([P0, P1, P2, P3, P4], dtype=float)

        if strict_bounds:
            _check_bounds_for_control_points(
                segment_controls,
                interval_index=i,
                lower_bound=lower_bound,
                upper_bound=upper_bound,
            )

        controls[i] = segment_controls

    return controls
```

This method is useful as a diagnostic reference because it shows what happens when the derivative conditions are imposed exactly.

5.6 Derivative Estimation

The exact and bounded quartic methods require derivative information at the support points. Since such derivatives are not directly measured, they are estimated numerically from the support values. The implementation allows two methods: ordinary finite differences and a PCHIP-like derivative estimate. The PCHIP-like method is used as the default because it reduces the risk of artificial oscillations.

Code Listing 8: Selection of the derivative estimation method.

```
def estimate_node_derivatives(x_nodes, y_nodes, method="pchip"):
    x_nodes, y_nodes = _validate_xy_nodes(x_nodes, y_nodes)

    if method == "pchip":
        return estimate_pchip_derivatives_numpy(x_nodes, y_nodes)

    if method == "finite_difference":
        return np.gradient(y_nodes, x_nodes)

    raise ValueError(
        "Unknown derivative_method. Use 'pchip' or 'finite_difference'."
    )
```

For an interior support point, the PCHIP-like method first computes the neighboring secant slopes. If the signs of two neighboring slopes are different, the derivative is set to zero. This prevents the construction from introducing

artificial extrema. If the slopes have the same sign, a weighted harmonic mean is used.

Code Listing 9: Interior PCHIP-like derivative estimation.

```
h = np.diff(x)
delta = np.diff(y) / h
d = np.zeros(n, dtype=float)

for i in range(1, n - 1):
    left = delta[i - 1]
    right = delta[i]

    if left == 0.0 or right == 0.0 or np.sign(left) != np.sign(right):
        d[i] = 0.0
    else:
        w1 = 2.0 * h[i] + h[i - 1]
        w2 = h[i] + 2.0 * h[i - 1]
        d[i] = (w1 + w2) / (w1 / left + w2 / right)
```

Thus, the derivative estimates are local and shape-preserving. In the exact quartic method they are imposed as hard constraints, while in the bounded quartic method they are only used as preferred reference slopes.

5.7 Construction of the Bounded Quartic Bézier Method

The bounded quartic method is the final constrained method. It keeps endpoint interpolation and mean preservation exact, but treats derivative information as soft. P_1 and P_3 are projected onto the feasible set, and P_2 is determined afterwards from the exact mean condition, if they were incompatible with the physical bounds.

Code Listing 10: Bounded quartic Bézier construction with soft derivative constraints.

```
def _compute_bounded_quartic_control_points(
    x_nodes,
    y_nodes,
    means,
    derivatives,
    lower_bound,
    upper_bound,
):
    n_intervals = len(x_nodes) - 1
```

```
    controls = np.zeros((n_intervals, 5), dtype=float)

    for i in range(n_intervals):
        h = x_nodes[i + 1] - x_nodes[i]

        y0 = y_nodes[i]
        y1 = y_nodes[i + 1]
        m = means[i]

        d0 = derivatives[i]
        d1 = derivatives[i + 1]

        P0 = y0
        P4 = y1

        internal_sum = 5.0 * m - P0 - P4

        P1_ref = P0 + (h / 4.0) * d0
        P3_ref = P4 - (h / 4.0) * d1

        P1, P3 = _project_p1_p3_to_bounded_mean_feasible_set(
            target_p1=P1_ref,
            target_p3=P3_ref,
            internal_sum=internal_sum,
            lower_bound=lower_bound,
            upper_bound=upper_bound,
            interval_index=i,
        )

        P2 = internal_sum - P1 - P3

        segment_controls = np.array([P0, P1, P2, P3, P4], dtype=float)

        _check_bounds_for_control_points(
            segment_controls,
            interval_index=i,
            lower_bound=lower_bound,
            upper_bound=upper_bound,
        )

        controls[i] = segment_controls

    return controls
```

The projection problem is two-dimensional because P_2 is determined by the mean condition once P_1 and P_3 are known. The feasible set is described by

$$L \le P_1 \le U, \qquad L \le P_3 \le U, \qquad L \le S_i - P_1 - P_3 \le U,$$

where

$$S_i = 5m_i - P_0 - P_4.$$

The implementation first checks whether such a feasible pair can exist.

Code Listing 11: Feasibility check for the bounded quartic projection problem.

```
lb = -np.inf if lower_bound is None else float(lower_bound)
ub = np.inf if upper_bound is None else float(upper_bound)

sum_low = -np.inf if upper_bound is None else internal_sum - ub
sum_high = np.inf if lower_bound is None else internal_sum - lb

min_possible_sum = 2.0 * lb
max_possible_sum = 2.0 * ub

feasible_sum_low = max(sum_low, min_possible_sum)
feasible_sum_high = min(sum_high, max_possible_sum)

if feasible_sum_low > feasible_sum_high:
    raise ValueError(
        f"Infeasible bounded quartic B\'ezier segment on interval {
  ↪ interval_index}: "
        f"endpoint values, exact mean and bounds cannot be satisfied
  ↪ simultaneously."
    )
```

After feasibility has been checked, candidates are generated from the reference point itself and from the boundary lines of the feasible set. The candidate with the smallest Euclidean distance to $(P_1^{\mathrm{ref}}, P_3^{\mathrm{ref}})$ is selected.

Code Listing 12: Selection of the closest feasible pair of internal quartic control points.

```
def add_candidate(p1, p3):
    p1 = float(p1)
    p3 = float(p3)

    if is_feasible(p1, p3):
        dist = (p1 - a) ** 2 + (p3 - b) ** 2
        candidates.append((dist, p1, p3))

add_candidate(a, b)
```

```
# Additional candidates are generated on the boundary lines
# P1 = lower/upper, P3 = lower/upper, and P1 + P3 = constant.

candidates.sort(key=lambda item: item[0])
_, best_p1, best_p3 = candidates[0]

return best_p1, best_p3
```

This construction preserves the endpoint values and interval means exactly. It also enforces the physical bounds whenever the bounded problem is feasible. The derivative information is retained as closely as possible, but it is not allowed to destroy physical admissibility.

5.8 Evaluation of the Piecewise Bézier Curves

After the control points have been computed, each Bézier curve is evaluated on a dense grid. The evaluation is performed locally on every support interval. For this purpose, each physical interval $[x_i, x_{i+1}]$ is mapped to the normalized local parameter interval $[0, 1]$.

In the implementation, the normalized parameter values are generated by `np.linspace`. They are then mapped back to the original time scale by

$$x = x_i + (x_{i+1} - x_i)t.$$

Code Listing 13: Evaluation of a piecewise Bézier curve on a normalized local grid.

```
for i in range(len(x_nodes) - 1):
    x0 = x_nodes[i]
    x1 = x_nodes[i + 1]

    if i < len(x_nodes) - 2:
        t = np.linspace(0.0, 1.0, num_points, endpoint=False)
    else:
        t = np.linspace(0.0, 1.0, num_points)

    x = x0 + (x1 - x0) * t
    y = evaluate_quartic_bezier_segment(controls[i], t)

    x_all.append(x)
    y_all.append(y)
```

The option `endpoint=False` is used on all intervals except the last one. This avoids duplicating support points at interval boundaries. The last interval includes its right endpoint, so the final support point is included in the evaluated curve.

When the curve is evaluated at a specific point x, the same normalization is used explicitly:

$$t = \frac{x - x_i}{x_{i+1} - x_i}.$$

This is also the normalization used in the Excel/VBA implementation.

5.9 Verification of Constraints

After construction and evaluation, the methods are checked numerically. The verification step is important because the formulas impose the constraints analytically, but the implementation must still be checked for numerical errors and possible bound violations.

For the cubic method, the verification routine checks endpoint interpolation and mean preservation. The maximum point error and maximum mean error are stored over all intervals.

Code Listing 14: Verification of endpoint and mean preservation for the cubic method.

```
err_left = abs(P[0] - y_nodes[i])
err_right = abs(P[3] - y_nodes[i + 1])

mean_val = np.sum(P) / 4.0
err_mean = abs(mean_val - means[i])

max_point_error = max(max_point_error, err_left, err_right)
max_mean_error = max(max_mean_error, err_mean)
```

For the exact quartic method, endpoint values, interval means, endpoint derivatives, and physical bounds are checked. The derivative errors are computed from the endpoint derivative formulas.

Code Listing 15: Verification of mean and derivative constraints for the exact quartic method.

```
mean_value = np.mean(P)
max_mean_error = max(max_mean_error, abs(mean_value - means[i]))

left_derivative = 4.0 * (P[1] - P[0]) / h
right_derivative = 4.0 * (P[4] - P[3]) / h

max_derivative_error = max(
    max_derivative_error,
    abs(left_derivative - derivatives[i]),
    abs(right_derivative - derivatives[i + 1]),
)
```

For the bounded quartic method, the derivatives are not hard constraints. Therefore, the verification routine reports the maximum derivative change instead of a derivative violation. This quantity measures how much the final curve had to deviate from the preferred derivative values in order to satisfy the mean and bound constraints.

Code Listing 16: Diagnostic derivative change for the bounded quartic method.

```
actual_left_derivative = 4.0 * (P[1] - P[0]) / h
actual_right_derivative = 4.0 * (P[4] - P[3]) / h

max_derivative_change = max(
    max_derivative_change,
    abs(actual_left_derivative - preferred_derivatives[i]),
    abs(actual_right_derivative - preferred_derivatives[i + 1]),
)
```

Physical bounds are checked directly on the control points. This is sufficient because the scalar Bézier curve lies inside the convex hull of its control points.

Code Listing 17: Verification of lower and upper bound violations.

```
if lower_bound is None:
    max_lower_violation = 0.0
else:
    max_lower_violation = float(
        np.max(np.maximum(lower_bound - controls, 0.0))
    )

if upper_bound is None:
    max_upper_violation = 0.0
else:
    max_upper_violation = float(
        np.max(np.maximum(controls - upper_bound, 0.0)))
```

The resulting verification report contains the maximum endpoint error, the maximum mean error, derivative error or derivative change, and possible lower or upper bound violations.

5.10 Error Analysis and Numerical Comparison

The interpolation methods are compared by evaluating their curves on a dense grid and comparing these values with the reference data. Since the reference measurements are available only at discrete timestamps, the reference values on the dense grid are obtained by linear interpolation.

Code Listing 18: Evaluation of reference values on the dense comparison grid.

```
x_plot, y_plot = evaluate_bounded_quartic_bezier(
    x_nodes=x_nodes,
    controls=controls,
    num_points=num_points,
)

y_true = np.interp(x_plot, x_ref, y_ref)
```

The residuals are defined as the difference between the reference values and the Bézier approximation. From these residuals, the script computes the root-mean-square error, mean absolute error, maximum absolute error, and the coefficient of determination.

Code Listing 19: Computation of approximation error metrics.

```
def compute_error_metrics(y_true, y_pred):
    residuals = y_true - y_pred

    rmse = float(np.sqrt(np.mean(residuals ** 2)))
    mae = float(np.mean(np.abs(residuals)))
    max_abs_error = float(np.max(np.abs(residuals)))

    ss_res = float(np.sum(residuals ** 2))
    ss_tot = float(np.sum((y_true - np.mean(y_true)) ** 2))
    r2 = np.nan if ss_tot == 0 else float(1.0 - ss_res / ss_tot)

    return {
        "RMSE": rmse,
        "MAE": mae,
        "MAX": max_abs_error,
        "R2": r2,
        "residuals": residuals,
    }
```

For each method, the evaluated curve, the computed error metrics, the constraint report, and the control points are stored in a dictionary. This makes it possible to print tables, plot all methods together, and compare the diagnostic quantities in a uniform way.

Code Listing 20: Storage of method results for comparison.

```
results[name] = {
    "x": x_plot,
    "y": y_plot,
    "metrics": compute_error_metrics(y_true, y_plot),
    "constraints": constraints,
    "controls": controls,
}
```

The comparison script applies this structure to three methods: the mean-preserving cubic Bézier method, the exact quartic Bézier method used as a diagnostic reference, and the bounded quartic Bézier method with soft derivative constraints.

5.11 Control-Polygon Roughness

In addition to the standard approximation errors, the bounded quartic method also computes the roughness of the control polygon. This quantity is not an interpolation constraint. It is only used as a diagnostic indicator of local bending or possible oscillatory behavior.

Code Listing 21: Computation of the control-polygon roughness.

```
def compute_control_polygon_roughness(controls):
    controls = np.asarray(controls, dtype=float)
    roughness = 0.0

    for P in controls:
        second_diff = P[:-2] - 2.0 * P[1:-1] + P[2:]
        roughness += float(np.sum(second_diff ** 2))

    return roughness
```

A larger roughness value indicates stronger variation in the control polygon. This can be useful when two methods have similar approximation errors but one of them produces a visibly less regular curve.

5.12 Graphical Diagnostics

The numerical comparison is complemented by graphical diagnostics. The first plot shows the reference data, the selected support points, and the Bézier curves. The second plot shows the residuals of the methods. This makes it possible to inspect not only the size of the error, but also where the errors occur.

Code Listing 22: Graphical comparison of the interpolation methods.

```
def plot_comparison(x_ref, y_ref, x_nodes, y_nodes, results):
    plt.figure(figsize=(14, 7))
    plt.plot(x_ref, y_ref, label="Reference data", linewidth=1.3, alpha
  ↪ =0.75)

    for name, info in results.items():
        if "error" in info:
            continue

        plt.plot(info["x"], info["y"], label=name, linewidth=2.0)

    plt.scatter(x_nodes, y_nodes, label="Support points", s=25, zorder=3)

    plt.xlabel("Time since start [hours]")
    plt.ylabel("Value")
    plt.title("Cubic mean B\'ezier vs quartic B\'ezier variants")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
```

The residual plot is especially useful for detecting systematic deviations. If the residuals are concentrated in specific intervals, this may indicate that the selected support step is too large or that the local dynamics are too complex for the chosen segment.

5.13 Excel/VBA Implementation

After the Python implementation had been tested, the bounded quartic Bézier method was translated into Excel/VBA. The purpose of this step was practical: hydrological and hydrochemical data are often stored and inspected in Excel, so the final method should be usable directly in a spreadsheet environment.

The VBA implementation follows the same main steps as the Python version.

It reads the original data from columns A and B, builds support points, computes empirical interval means, checks feasibility, constructs bounded quartic Bézier control points, evaluates the interpolated curve, writes the results back to the worksheet.

timestamp	value	Support row	Interval	x_start	x_end	true mean	P0	P1	P2	P3	P4	timestamp	real value	Bezier interpolated value
Mon/ 01/01/2024 00:00	78	2	0	01.01.2024 00:00	08.01.2024 00:00	215.0044577	78	114.375	600.6472883	141	141	01.01.2024 00:00	78	78
Mon/ 01/01/2024 00:15	78	674	1	08.01.2024 00:00	15.01.2024 00:00	62.47251114	141	114.2986692	0	18.06388656	39	01.01.2024 00:15	78	78.22247706
Mon/ 01/01/2024 00:30	78	1346	2	15.01.2024 00:00	22.01.2024 00:00	31.96433878	39	33.23478261	35.5869113	26	26	01.01.2024 00:30	78	78.4567991
Mon/ 01/01/2024 00:45	78	2018	3	22.01.2024 00:00	29.01.2024 00:00	95.78157504	26	26	274.9078752	76	76	01.01.2024 00:45	78	78.70285623
Mon/ 01/01/2024 01:00	78	2690	4	29.01.2024 00:00	05.02.2024 00:00	50.57057949	76	76	14.85289747	43	43	01.01.2024 01:00	78	78.96053888
Mon/ 01/01/2024 01:15	78	3362	5	05.02.2024 00:00	12.02.2024 00:00	155.6582467	43	43	214.2912333	239	239	01.01.2024 01:15	78	79.2297378
Mon/ 01/01/2024 01:30	78	4034	6	12.02.2024 00:00	19.02.2024 00:00	95.3640416	239	181.820208	0	0	56	01.01.2024 01:30	78	79.5103441
Mon/ 01/01/2024 01:45	78	4706	7	19.02.2024 00:00	26.02.2024 00:00	75.83655275	56	56	103.1827637	82	82	01.01.2024 01:45	78	79.8022492
Mon/ 01/01/2024 02:00	78	5378	8	26.02.2024 00:00	04.03.2024 00:00	52.35661218	82	82	23.6602539	39.12280702	35	01.01.2024 02:00	78	80.10534485
Mon/ 01/01/2024 02:15	77	6050	9	04.03.2024 00:00	11.03.2024 00:00	28.8127786	35	30.87719298	28.18670003	25	25	01.01.2024 02:15	77	80.41952313
Mon/ 01/01/2024 02:30	77	6722	10	11.03.2024 00:00	18.03.2024 00:00	31.08320951	25	25	44.91604755	29.5	31	01.01.2024 02:30	77	80.74467646
Mon/ 01/01/2024 02:45	77	7394	11	18.03.2024 00:00	25.03.2024 00:00	30.53640416	31	32.5	17.2820208	34.9	37	01.01.2024 02:45	77	81.08069757
Mon/ 01/01/2024 03:00	77	8066	12	25.03.2024 00:00	01.04.2024 00:00	43.06686478	37	39.1	38.78987948	49.44444444	51	01.01.2024 03:00	77	81.42747954
Mon/ 01/01/2024 03:15	77	8738	13	01.04.2024 00:00	08.04.2024 00:00	58.25705795	51	52.55555556	77.72973419	55	55	01.01.2024 03:15	77	81.78491576
Mon/ 01/01/2024 03:30	77	9410	14	08.04.2024 00:00	15.04.2024 00:00	36.43090639	55	55	22.15453195	25	25	01.01.2024 03:30	77	82.15289997
Mon/ 01/01/2024 03:45	77	10082	15	15.04.2024 00:00	22.04.2024 00:00	33.33580981	25	25	32.67904903	42	42	01.01.2024 03:45	77	82.53132623
Mon/ 01/01/2024 04:00	77	10754	16	22.04.2024 00:00	29.04.2024 00:00	33.58098068	42	42	33.90490342	25	25	01.01.2024 04:00	77	82.92008893
Mon/ 01/01/2024 04:15	77	11426	17	29.04.2024 00:00	06.05.2024 00:00	29.41604755	25	25	36.08023774	30	31	01.01.2024 04:15	77	83.31908279
Mon/ 01/01/2024 04:30	76	12098	18	06.05.2024 00:00	13.05.2024 00:00	61.79049034	31	32	177.9524517	34	34	01.01.2024 04:30	76	83.72820285
Mon/ 01/01/2024 04:45	76	12770	19	13.05.2024 00:00	20.05.2024 00:00	26.79494799	34	34	5.97473997	30	30	01.01.2024 04:45	76	84.1473445
Mon/ 01/01/2024 05:00	76	13442	20	20.05.2024 00:00	27.05.2024 00:00	38.79197623	30	30	65.9390478	33.02083333	35	01.01.2024 05:00	76	84.57640345
Mon/ 01/01/2024 05:15	76	14114	21	27.05.2024 00:00	03.06.2024 00:00	39.11589896	35	36.97916667	15.60032813	54	54	01.01.2024 05:15	76	85.01527573
Mon/ 01/01/2024 05:30	76	14786	22	03.06.2024 00:00	10.06.2024 00:00	33.76077266	54	54	19.42548492	21.37837838	20	01.01.2024 05:30	76	85.46385771
Mon/ 01/01/2024 05:45	76	15458	23	10.06.2024 00:00	17.06.2024 00:00	18.15156018	20	18.62162162	17.76117927	17.375	17	01.01.2024 05:45	76	85.92204609
Mon/ 01/01/2024 06:00	76	16130	24	17.06.2024 00:00	24.06.2024 00:00	18.4205052	17	16.625	26.477526	16	16	01.01.2024 06:00	76	86.3897379
Mon/ 01/01/2024 06:15	75	16802	25	24.06.2024 00:00	01.07.2024 00:00	18.72065379	16	0	0	26.60326895	51	01.01.2024 06:15	75	86.86683049
Mon/ 01/01/2024 06:30	75	17474	26	01.07.2024 00:00	08.07.2024 00:00	26.84249629	51	48.63481214	0	16.57766928	18	01.01.2024 06:30	75	87.35322156
Mon/ 01/01/2024 06:45	75	18146	27	08.07.2024 00:00	15.07.2024 00:00	19.04309064	18	17.05714286	27.49164367	16.66666667	16	01.01.2024 06:45	75	87.84880911
Mon/ 01/01/2024 07:00	75	18818	28	15.07.2024 00:00	22.07.2024 00:00	13.22585438	16	15.33333333	10.79593858	12	12	01.01.2024 07:00	75	88.35349149
Mon/ 01/01/2024 07:15	75	19490	29	22.07.2024 00:00	29.07.2024 00:00	16.72065379	12	9.801634473	0	29.80163447	32	01.01.2024 07:15	75	88.86716739
Mon/ 01/01/2024 07:30	75	20162	30	29.07.2024 00:00	05.08.2024 00:00	17.37147103	32	29.60259061	0	12.25476452	13	01.01.2024 07:30	75	89.3897358
Mon/ 01/01/2024 07:45	75	20834	31	05.08.2024 00:00	12.08.2024 00:00	11.87964339	13	11.34782609	17.05039085	9	9	01.01.2024 07:45	75	89.92109606
Mon/ 01/01/2024 08:00	75	21506	32	12.08.2024 00:00	19.08.2024 00:00	9.901931649	9	9	11.50965825	10	10	01.01.2024 08:00	75	90.46114783
Mon/ 01/01/2024 08:15	75	22178	33	19.08.2024 00:00	26.08.2024 00:00	8.88410104	10	10	8.087171867	8.333333333	8	01.01.2024 08:15	75	91.00979112
Mon/ 01/01/2024 08:30	75	22850	34	26.08.2024 00:00	02.09.2024 00:00	7.50371471	8	7.666666667	7.851906885	7	7	01.01.2024 08:30	75	91.56692624
Mon/ 01/01/2024 08:45	74	23522	35	02.09.2024 00:00	09.09.2024 00:00	8.013372957	7	7	0.066864785	13	13	01.01.2024 08:45	74	92.13245386
Mon/ 01/01/2024 09:00	74	24194	36	09.09.2024 00:00	16.09.2024 00:00	11.0282318	13	13	12.72449232	8.416666667	8	01.01.2024 09:00	74	92.70627494
Mon/ 01/01/2024 09:15	74	24866	37	16.09.2024 00:00	23.09.2024 00:00	7.589895988	8	7.583333333	8.366146607	7	7	01.01.2024 09:15	74	93.28829081
Mon/ 01/01/2024 09:30	74	25538	38	23.09.2024 00:00	30.09.2024 00:00	21.25111441	7	7	56.25557207	18	18	01.01.2024 09:30	74	93.87840311
Mon/ 01/01/2024 09:45	74	26210	39	30.09.2024 00:00	07.10.2024 00:00	18.19167905	18	18	24.95839525	15	15	01.01.2024 09:45	74	94.47651381
Mon/ 01/01/2024 10:00	74	26882	40	07.10.2024 00:00	14.10.2024 00:00	39.91827637	15	15	89.59138187	40	40	01.01.2024 10:00	74	95.08252522
Mon/ 01/01/2024 10:15	74	27554	41	14.10.2024 00:00	21.10.2024 00:00	25.81129272	40	40	8.723130263	21.33333333	19	01.01.2024 10:15	74	95.69633997

Figure 2: Example of a final table

Then it creates a comparison chart for visualization of the interpolation results.

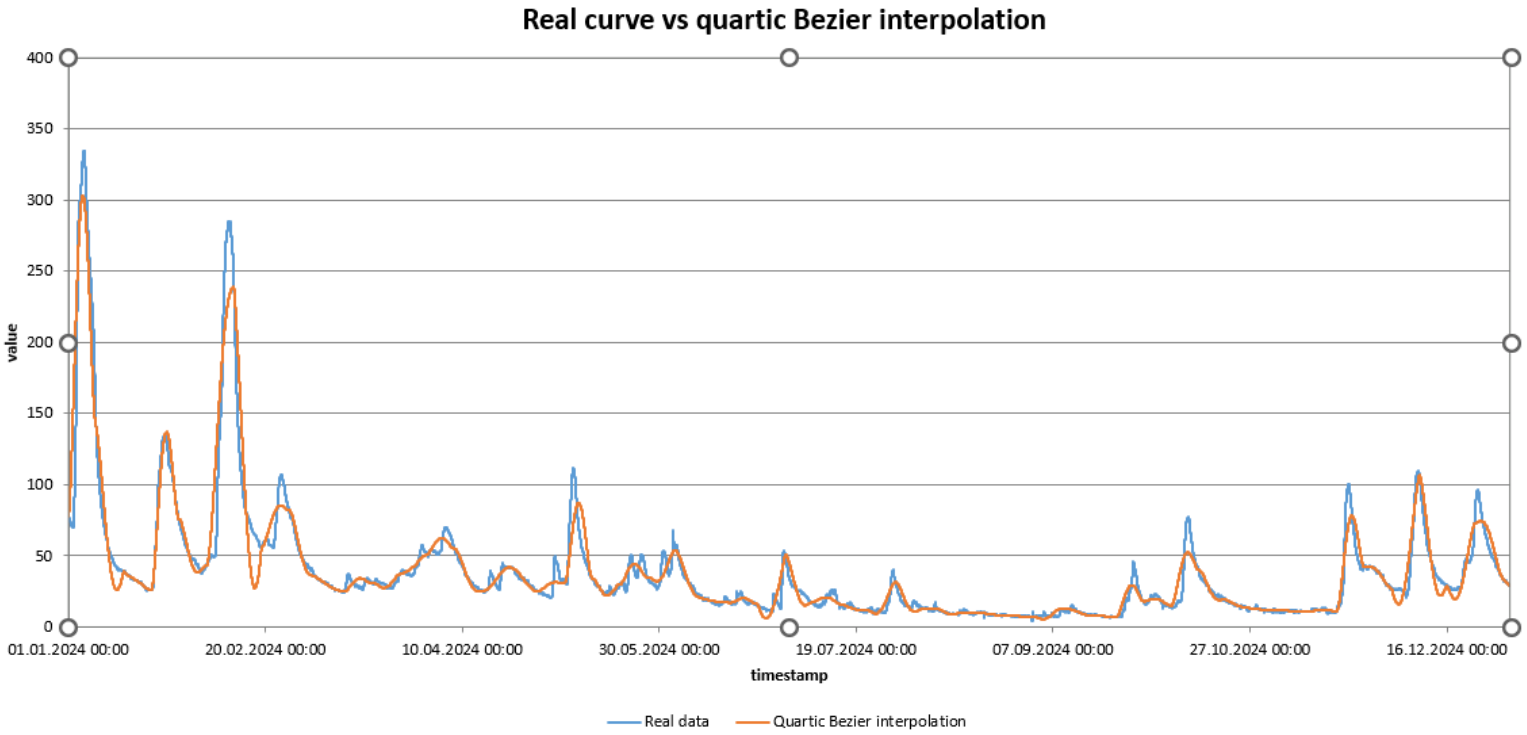


Figure 3: Interpolated curve compared with measured data.

The data input stage reads the original timestamps and values from the worksheet and stores them in arrays.

Code Listing 23: Reading original data from the Excel worksheet.

```
Dim rawX() As Double
Dim rawY() As Double
ReDim rawX(0 To nRaw - 1)
ReDim rawY(0 To nRaw - 1)

For i = 0 To nRaw - 1

    If IsEmpty(ws.Cells(DATA_FIRST_ROW + i, COL_X).Value) Or _
       IsEmpty(ws.Cells(DATA_FIRST_ROW + i, COL_Y).Value) Then
        MsgBox "Empty cell found in row " & (DATA_FIRST_ROW + i) & ".",
  ↪ vbCritical
        Exit Sub
    End If
    rawX(i) = CDbl(ws.Cells(DATA_FIRST_ROW + i, COL_X).Value)
    rawY(i) = CDbl(ws.Cells(DATA_FIRST_ROW + i, COL_Y).Value)

Next i
```

Support indices are then created using the prescribed value `SUPPORT_STEP`. As in Python, the last data point is added if it is not already part of the support set.

Code Listing 24: Construction of support point indices in VBA.

```
For i = 0 To nRaw - 1 Step SUPPORT_STEP
    supportCount = supportCount + 1
Next i

If (nRaw - 1) Mod SUPPORT_STEP <> 0 Then
    supportCount = supportCount + 1
End If

ReDim supportIndex(0 To supportCount - 1)

k = 0

For i = 0 To nRaw - 1 Step SUPPORT_STEP
    supportIndex(k) = i
    k = k + 1
Next i

If supportIndex(k - 1) <> nRaw - 1 Then
    supportIndex(k) = nRaw - 1
End If
```

The empirical means are computed in the same way as in Python: all original values between two consecutive support indices are summed and divided by their number.

Code Listing 25: Computation of empirical interval means in VBA.

```
For i = 0 To supportCount - 2

    startIdx = supportIndex(i)
    endIdx = supportIndex(i + 1)

    sumY = 0#
    countY = 0

    For j = startIdx To endIdx
        sumY = sumY + rawY(j)
        countY = countY + 1
    Next j

    means(i) = sumY / countY

Next i
```

Before constructing the bounded quartic curve, the VBA implementation checks whether the exact mean and lower bound are compatible. In the case $L = 0$, a necessary feasibility condition is

$$m_i \geq \frac{y_i + y_{i+1}}{5}.$$

Code Listing 26: Feasibility check for the bounded quartic construction in VBA.

```
For i = 0 To supportCount - 2

    minimalMean = (yNodes(i) + yNodes(i + 1)) / 5#

    If means(i) < minimalMean Then
        MsgBox _
            "Problem on interval " & i & vbCrLf & _
            "true interval mean = " & means(i) & vbCrLf & _
            "minimal possible mean with lower bound 0 = " & minimalMean &
  ↪ vbCrLf & _
            "This interval is infeasible.", _
            vbCritical
        Exit Sub
    End If

Next i
```

The bounded quartic control points are then constructed by the VBA function `BuildBoundedQuarticBezierControls`, which internally estimates PCHIP-like derivatives and applies the same bounded quartic projection logic as the Python version.

Code Listing 27: Calling the bounded quartic Bézier construction in VBA.

```
controls = BuildBoundedQuarticBezierControls( _
    xNodes, _
    yNodes, _
    means, _
    LOWER_BOUND, _
    USE_UPPER_BOUND, _
    UPPER_BOUND _
)
```

For the evaluation at a given worksheet timestamp, the corresponding interval is found and the local normalized parameter t is computed as

$$t = \frac{x - x_i}{x_{i+1} - x_i}.$$

Code Listing 28: Evaluation of the piecewise quartic Bézier curve at a worksheet timestamp.

```
If xValue >= xNodes(i) And xValue <= xNodes(i + 1) Then

    t = (xValue - xNodes(i)) / (xNodes(i + 1) - xNodes(i))

    EvaluatePiecewiseQuarticBezierAtX = EvaluateQuarticBezierSegment( _
        controls(i, 0), controls(i, 1), controls(i, 2), _
        controls(i, 3), controls(i, 4), t _
    )

    Exit Function

End If
```

The computed support intervals, control points, evaluated interpolation, and verification values are written back into the worksheet. The main worksheet layout is summarized in Table 1.

Worksheet range	Content
A:B	Original timestamps and measured values
D:M	Support intervals and Bézier control points
O:Q	Original values and evaluated interpolation
S:T	Verification quantities
Chart area	Visual comparison of measured data and interpolation

Table 1: Main worksheet input and output ranges used by the Excel/VBA implementation.

The VBA implementation is therefore not a separate mathematical method. It is an operational translation of the bounded quartic Bézier construction into Excel.

5.14 Summary of the Implementation Logic

The complete implementation can be summarized as follows:

1. The original measured data are loaded from Excel or from an Excel file.
2. Timestamps are converted into a numerical time variable.
3. Support points are selected using a fixed step size.
4. Empirical interval means are computed from all original values inside each support interval.
5. Cubic, exact quartic, and bounded quartic Bézier control points are constructed.
6. The exact quartic method uses derivative estimates as hard constraints.
7. The bounded quartic method uses derivative estimates only as soft reference slopes.
8. Physical bounds are enforced through the Bézier control points.
9. The piecewise curves are evaluated using the normalized local parameter $t \in [0, 1]$.
10. Constraint errors, bound violations, derivative errors or derivative changes, and roughness values are computed.
11. The approximations are compared with the original reference data using RMSE, MAE, maximum error, and R^2.
12. The curves and residuals are plotted for visual inspection.

This organization separates the mathematical construction from the practical workflow.

5.15 Source Codes

The complete Python and VBA implementations used in this work are available in the accompanying project repository. The Python implementation comprises the mean-preserving cubic Bézier method, the exact quartic Bézier method, and the bounded quartic Bézier method with soft derivative constraints. A separate comparison script performs the numerical evaluation of the three methods. The final bounded quartic method is additionally implemented in Excel/VBA for use in a spreadsheet-based workflow.

Selected implementation steps are reproduced in the code listings throughout this section, while the complete source files are provided separately in the project repository.

6 Numerical Results and Comparison

This section presents the numerical comparison of the three Bézier-based interpolation methods considered in this work. The comparison is based on approximation accuracy, preservation of the imposed constraints, physical admissibility, and visual behavior of the final curves.

The results must be interpreted with respect to two different aspects. First, the interpolating curve should approximate the reference data accurately. Second, it must satisfy the prescribed mathematical and physical constraints. Therefore, a method with good error metrics cannot be considered fully satisfactory if it violates the physical bound or does not preserve the interval means.

6.1 Numerical Setup

The numerical experiment was performed on a reference data set containing 35136 measured values. Support points were selected with a fixed step size of 672, resulting in 54 support points and 53 interpolation intervals. The interval means were computed as sample averages of the original reference values within each support interval. The lower physical bound was set to zero, while no upper bound was imposed. For the quartic methods, the derivative values were estimated using the PCHIP-like derivative estimator.

Quantity	Value
Number of reference points	35136
Number of support points	54
Number of interpolation intervals	53
Support step size	672
Mean computation mode	Sample average
Lower physical bound	0.0
Upper physical bound	Not imposed
Quartic derivative method	PCHIP-like

Table 2: Numerical setup used for the comparison of the interpolation methods.

This information is important for reproducibility, since the number of support points, the support step size, and the imposed physical bounds directly influ-

ence the interpolation result. In particular, the support step size determines how much information is available to reconstruct the behavior between two neighboring support points.

6.2 Approximation Quality

The first part of the comparison concerns approximation quality. Each interpolating curve was evaluated on a dense grid and compared with the reference data. The considered error metrics are the root-mean-square error RMSE, the mean absolute error MAE, the maximum absolute error, and the coefficient of determination R^2. In addition, the control-polygon roughness is reported as a diagnostic indicator of local bending and possible oscillatory behavior.

Method	RMSE	MAE	Max. error	R^2	Roughness
Cubic Bézier baseline, mean-preserving	12.007279	5.330826	113.560781	0.921061	1.631647×10^5
Quartic Bézier, mean and hard derivatives	11.242047	5.560609	94.711293	0.930802	2.768685×10^6
Bounded quartic Bézier with soft derivatives	10.947902	5.470358	94.711293	0.934376	2.136015×10^6

Table 3: Approximation quality and control-polygon roughness of the considered Bézier interpolation methods.

Table 3 shows that no single method dominates all diagnostic criteria. The bounded quartic method achieves the smallest RMSE, 10.947902, and the largest coefficient of determination, $R^2 = 0.934376$. Relative to the cubic baseline, its RMSE is lower by approximately 8.8%, while R^2 increases by approximately 0.0133.

The cubic method, however, gives the smallest MAE, 5.330826, compared with 5.470358 for the bounded quartic method. Thus, the bounded quartic method reduces the squared-error-dominated RMSE while the cubic method retains a slightly smaller average absolute deviation. The bounded quartic MAE is approximately 2.6% larger than the cubic MAE.

The maximum absolute error also reveals a trade-off. Both quartic methods have the same maximum error of 94.711293, which is lower than the cubic value of 113.560781. This indicates that the quartic constructions handle the largest local deviations more effectively for the chosen support spacing.

The roughness diagnostic leads to a different conclusion. The cubic construction has by far the smallest control-polygon roughness, 1.631647×10^5. The

exact quartic and bounded quartic values are 2.768685×10^6 and 2.136015×10^6, respectively. Consequently, the cubic construction is substantially smoother according to this discrete control-polygon diagnostic, whereas the quartic methods trade part of this smoothness for greater local flexibility.

The exact quartic method lies between the cubic and bounded quartic methods in terms of RMSE and R^2, but its MAE is the largest of the three. Its role in the comparison is therefore especially useful as a diagnostic reference for the effect of enforcing endpoint derivatives exactly.

6.3 Constraint Verification

The second part of the comparison concerns the imposed constraints.

Method	Endpoint error	Mean error	Control lower violation	Dense minimum	Dense lower violation
Cubic Bézier baseline	0	1.421085×10^{-14}	0	6.265012	0
Quartic Bézier	0	2.842171×10^{-14}	1.131798×10^{2}	2.709591	0
Bounded quartic Bézier	0	2.842171×10^{-14}	0	5.605888	0

Table 4: Verification of endpoint interpolation, mean preservation, and lower-bound admissibility. The control lower violation is computed from the Bézier control points, while the dense lower violation is computed from the evaluated curve on the dense grid.

Table 4 shows that all methods preserve the endpoint values exactly. The endpoint error is zero for all three methods, which is expected because the first and last control points of each Bézier segment are chosen to be the corresponding support values.

The interval mean is also preserved up to numerical precision. For the cubic method, the maximum mean error is 1.421085×10^{-14}. For the quartic methods, the maximum mean error is 2.842171×10^{-14}. These values are negligible and can be interpreted as floating-point round-off error. Therefore, the mean-preserving condition is successfully enforced by all three constructions.

The main difference between the methods appears in the lower-bound verification. The exact quartic method has a control lower violation of 1.131798×10^2. This means that at least one control point lies below the prescribed lower bound. The evaluated dense grid still has a positive minimum value, so the sampled curve does not become negative in this experiment. However, the convex-hull guarantee is lost: bounded control points are a sufficient condition

for the full scalar Bézier curve to remain bounded. If a control point violates the bound, this guarantee no longer holds.

This violation is not accidental. In the exact quartic formulation, the endpoint values, the interval mean, and both endpoint derivatives are imposed as hard constraints. These five conditions uniquely determine the five control points of the quartic Bézier segment. Consequently, there is no remaining degree of freedom to adjust the control polygon if the resulting control points violate the physical lower bound.

In contrast, the bounded quartic Bézier method avoids this issue. It preserves endpoint values and interval means up to numerical precision and has no control lower violation. This is achieved by treating the derivative estimates as soft reference slopes. The derivative information is used to define preferred control points, but these points are modified whenever necessary to preserve the lower bound and the exact mean constraint.

6.4 Derivative Information

The exact and bounded quartic methods both use derivative information, but in different ways. In the exact quartic method, the derivatives are imposed as hard constraints. The maximum derivative error is

$$1.249001 \times 10^{-16},$$

which is essentially zero. This confirms that the derivative constraints are implemented correctly. However, the same exact enforcement is also the reason why physical bounds may become impossible to satisfy.

In the bounded quartic method, the derivative estimates are not imposed unconditionally. Instead, they are treated as preferred reference slopes. The maximum derivative change is

$$1.361424.$$

This value should not be interpreted as an interpolation error. It measures how much the final bounded curve had to deviate from the preferred derivative information in order to preserve endpoint interpolation, exact interval means, and physical admissibility. Thus, the bounded quartic method deliberately

sacrifices exact derivative matching when the derivative constraints conflict with the physical lower bound.

6.5 Graphical Comparison

The numerical tables are complemented by graphical diagnostics. The first graph compares the original reference data, the selected support points, and the three Bézier interpolation methods. The second graph shows the residuals, defined as

$$\text{residual} = \text{reference value} - \text{approximated value}.$$

Thus, a positive residual means that the interpolation underestimates the reference data, while a negative residual means that it overestimates them.

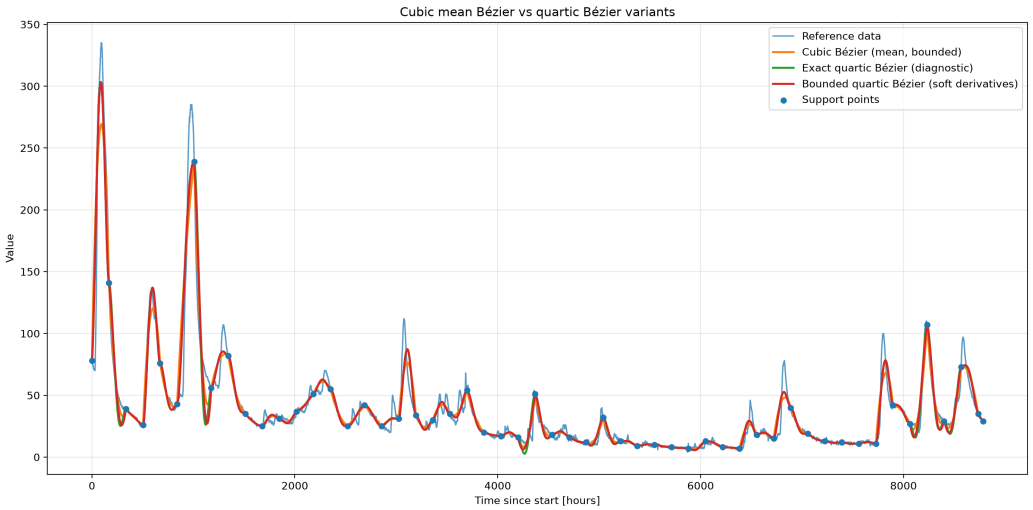


Figure 4: Comparison of the reference data, support points, and the three Bézier interpolation methods.

Figure 4 shows that all three methods reproduce the general structure of the reference data. The largest difficulties occur near sharp peaks and rapid transitions. This is expected because the support points are relatively sparse compared with the full reference data set, so the interpolating curves must reconstruct local behavior over long support intervals.

The cubic Bézier baseline produces the smoothest control structure and follows the overall trend well. The quartic methods have one additional control point and use derivative information, which gives them greater local flexibility. This flexibility reduces RMSE and the largest local error, but it also increases the control-polygon roughness relative to the cubic method.

The exact quartic method cannot be selected solely on visual accuracy. Table 4 shows that it violates the lower-bound condition at the level of the control points. Therefore, it does not provide the required convex-hull-based physical guarantee. The bounded quartic method retains the local flexibility of the quartic formulation while correcting the internal control points whenever necessary to preserve physical admissibility.

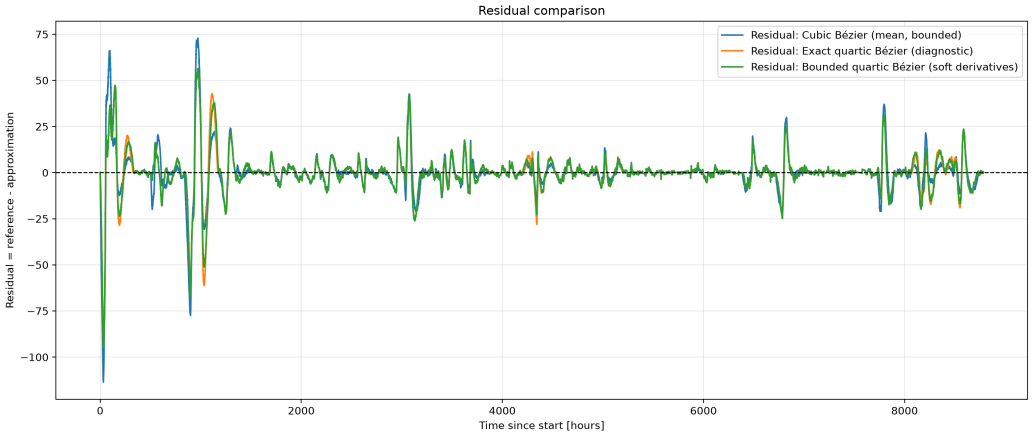


Figure 5: Residual comparison of the three interpolation methods. The residual is defined as reference value minus approximated value.

Figure 5 gives a more detailed view of the local errors. The largest residuals occur near sharp peaks and rapid changes, especially in the first part of the time series. The bounded quartic method reduces the squared-error contribution of the larger deviations sufficiently to obtain the smallest RMSE, while the cubic method has the smallest MAE. Thus, the residual comparison confirms that the methods emphasize different aspects of approximation quality rather than producing a strict ranking under all metrics.

6.6 Overall Interpretation

The comparison reveals a clear trade-off between smoothness, derivative fidelity, approximation accuracy, and physical admissibility.

The cubic Bézier method is the simplest and smoothest of the three according to the control-polygon roughness diagnostic. It preserves endpoint values, interval means, and the lower bound, and it achieves the smallest MAE. Its main disadvantage in this experiment is the larger RMSE and maximum absolute error.

The exact quartic method improves RMSE, maximum error, and R^2 relative to the cubic method and reproduces the prescribed endpoint derivatives almost exactly. However, its control points violate the lower bound, so the convex-hull-based boundedness guarantee is lost.

The bounded quartic method gives the smallest RMSE and the largest R^2, matches the lower maximum error of the exact quartic method, preserves endpoint values and interval means, and maintains the lower-bound guarantee. It does this by allowing derivative values to change when exact derivative matching is incompatible with the physical constraint.

For the present application, in which physical admissibility is mandatory and large local deviations are important, the bounded quartic method therefore provides the most appropriate compromise. The corrected cubic method remains a strong alternative when simplicity, low average absolute error, and a smoother control polygon are prioritized.

7 Conclusion

This work investigated Bézier-based interpolation methods for the approximation of hydrochemical and hydrological time-series data under simultaneous mathematical and physical constraints.

Three Bézier-based methods were developed and compared. The mean-preserving cubic construction enforces endpoint values, interval means, and physical bounds while selecting the unconstrained internal control points by minimizing a second-derivative smoothness functional. The exact quartic construction adds hard endpoint-derivative constraints. The bounded quartic construction instead treats those derivatives as soft targets so that physical bounds can still be enforced.

The corrected numerical comparison does not produce one method that is best under every metric. The cubic method achieves the smallest MAE, 5.330826, and by far the smallest control-polygon roughness, 1.631647×10^5. The bounded quartic method achieves the smallest RMSE, 10.947902, and the largest coefficient of determination, $R^2 = 0.934376$. Both quartic methods reduce the maximum absolute error from 113.560781 for the cubic method to 94.711293.

The exact quartic method demonstrates the main structural limitation of hard derivative constraints. Although it matches the prescribed endpoint derivatives to numerical precision, its control polygon violates the lower bound by 1.131798×10^2. The evaluated curve remains positive on the chosen dense grid, but the convex-hull-based boundedness guarantee is lost.

The bounded quartic formulation resolves this conflict by modifying the preferred derivative information only when necessary. In the reported experiment, its maximum derivative change is 1.361424, while endpoint values and interval means remain satisfied up to numerical precision and the control points satisfy the lower bound.

These results show that method selection depends on the priorities of the application. The cubic construction is attractive when simplicity, small average absolute error, and low roughness are especially important. The bounded quartic method is preferable when physical admissibility must be guaranteed while reducing larger local deviations and retaining derivative-informed local flexibility. For the present application, this combination makes the bounded quartic method the preferred final method.

The graphical comparison and residual analysis show that the largest errors are concentrated near sharp peaks and rapid transitions. This is expected because the selected support points are sparse relative to the full reference series. Consequently, the quality of the reconstruction remains sensitive to support-point spacing and derivative estimation.

The methods are implemented and tested in Python, and the final bounded quartic method is additionally available in Excel/VBA for spreadsheet-based use. Automated tests verify the principal interpolation constraints and are executed in continuous integration.

Several limitations remain. The prescribed interval means in the present study are arithmetic averages of the measured values. For irregularly spaced data, a time-weighted mean would be more appropriate. Only a lower physical bound was active in the numerical experiment, whereas future applications may require both lower and upper bounds. Further work could therefore investigate adaptive support-point selection, time-weighted mean preservation, improved derivative estimation, and automatic identification of intervals in which the constraints compete most strongly.

References

[1] Albrecht Gnauck. “Interpolation and Approximation of Water Quality Time Series and Process Identification”. In: *Analytical and Bioanalytical Chemistry* 380.3 (2004), pp. 484–492. DOI: 10.1007/s00216-004-2799-3. URL: https://pubmed.ncbi.nlm.nih.gov/15365680/ (visited on 06/18/2026).

[2] Mathieu Lepot, Jean-Baptiste Aubin, and Francois H. L. R. Clemens. “Interpolation in Time Series: An Introductive Overview of Existing Methods, Their Performance Criteria and Uncertainty Assessment”. In: *Water* 9.10 (2017), p. 796. DOI: 10.3390/w9100796. URL: https://www.mdpi.com/2073-4441/9/10/796 (visited on 06/18/2026).

[3] S. S. Kokhan, A. A. Moskalenko, and L. H. Shylo. “Geoinformation Support for Soil Quality Assessment”. In: *Eastern-European Journal of Enterprise Technologies* 6.3(66) (2013), pp. 18–25. DOI: 10.15587/1729-4061.2013.19174. URL: https://journals.uran.ua/eejet/article/view/19174 (visited on 06/18/2026).

[4] A. V. Piatkova, O. B. Murkalov, and Yu. V. Lohvyna. “Influence of the Digital Elevation Model Construction Method on Soil Erosion Calculation Results”. In: *Bulletin of Odessa National University. Geography and Geology* 24.2(35) (2019). DOI: 10.18524/2303-9914.2019.2(35).183729. URL: https://visgeo.onu.edu.ua/article/view/183729 (visited on 06/18/2026).

[5] L. V. Krylyk, I. V. Bohach, and M. O. Prokopova. *Computational Mathematics: Interpolation and Approximation of Tabular Data. Textbook*. Vinnytsia: Vinnytsia National Technical University, 2013. 111 pp. URL: https://ir.lib.vntu.edu.ua/handle/123456789/34695 (visited on 06/18/2026).

[6] Carl de Boor. *A Practical Guide to Splines*. New York: Springer, 1978. URL: https://archive.org/details/practicalguideto0027debo (visited on 06/18/2026).

[7] F. N. Fritsch and R. E. Carlson. “Monotone Piecewise Cubic Interpolation”. In: *SIAM Journal on Numerical Analysis* 17.2 (1980), pp. 238–246. DOI: 10.1137/0717021. URL: https://epubs.siam.org/doi/10.1137/0717021 (visited on 06/18/2026).

[8] F. N. Fritsch and J. Butland. “A Method for Constructing Local Monotone Piecewise Cubic Interpolants”. In: *SIAM Journal on Scientific and Statistical Computing* 5.2 (1984), pp. 300–304. DOI: 10.1137/0905021. URL: https://epubs.siam.org/doi/10.1137/0905021 (visited on 06/18/2026).

[9] SciPy Developers. *PchipInterpolator*. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.PchipInterpolator.html (visited on 06/18/2026).

[10] Gerald Farin. *Curves and Surfaces for CAGD: A Practical Guide*. 5th ed. Morgan Kaufmann, 2002. ISBN: 9781558607378. URL: https://shop.elsevier.com/books/curves-and-surfaces-for-cagd/farin/978-1-55860-737-8 (visited on 06/18/2026).

[11] Les Piegl and Wayne Tiller. *The NURBS Book*. 2nd ed. Berlin and Heidelberg: Springer, 1997. DOI: 10.1007/978-3-642-59223-2. URL: https://link.springer.com/book/10.1007/978-3-642-59223-2 (visited on 06/18/2026).

[12] Nicholas M. Patrikalakis, Takashi Maekawa, and Wonjoon Cho. *Definition of Bézier Curve and Its Properties*. 2009. URL: https://web.mit.edu/hyperbook/Patrikalakis-Maekawa-Cho/node12.html (visited on 06/18/2026).

[13] Aalto University. *Bézier Curves*. 2013. URL: https://math.aalto.fi/~ahniemi/hss2013/Section05.pdf (visited on 06/18/2026).

[14] Pomax. *A Primer on Bézier Curves*. URL: https://pomax.github.io/bezierinfo/ (visited on 06/18/2026).

[15] Tianfeng Chai and Roland R. Draxler. “Root Mean Square Error (RMSE) or Mean Absolute Error (MAE)? Arguments against Avoiding RMSE in the Literature”. In: *Geoscientific Model Development* 7 (2014), pp. 1247–1250. DOI: 10.5194/gmd-7-1247-2014. URL: https://gmd.copernicus.org/articles/7/1247/2014/ (visited on 06/18/2026).

[16] Cort J. Willmott and Kenji Matsuura. “Advantages of the Mean Absolute Error (MAE) over the Root Mean Square Error (RMSE) in Assessing Average Model Performance”. In: *Climate Research* 30.1 (2005), pp. 79–82. DOI: 10.3354/cr030079. URL: https://www.int-res.com/journals/cr/articles/cr030079 (visited on 06/18/2026).

List of Figures

1 Example of a table 19

2 Example of a final table 34

3 Interpolated curve compared with measured data. 35

4 Comparison of the reference data, support points, and the three Bézier interpolation methods. 45

5 Residual comparison of the three interpolation methods. The residual is defined as reference value minus approximated value. 46

List of Tables

1 Main worksheet input and output ranges used by the Excel/VBA implementation. 38

2 Numerical setup used for the comparison of the interpolation methods. 41

3 Approximation quality and control-polygon roughness of the considered Bézier interpolation methods. 42

4 Verification of endpoint interpolation, mean preservation, and lower-bound admissibility. The control lower violation is computed from the Bézier control points, while the dense lower violation is computed from the evaluated curve on the dense grid. 43

List of Code Listings

1 Main structure of the Python comparison workflow. 18
2 Loading and preprocessing of the reference data. 20
3 Selection of support points from the reference data. 20
4 Computation of empirical interval means from reference data. 21
5 Construction of cubic Bézier control points with exact mean preservation. 21
6 Projection of the cubic internal control points onto the bounded mean-preserving feasible set. 22
7 Exact quartic Bézier construction with endpoint, mean, and derivative constraints. 23
8 Selection of the derivative estimation method. 24
9 Interior PCHIP-like derivative estimation. 25
10 Bounded quartic Bézier construction with soft derivative constraints. 25
11 Feasibility check for the bounded quartic projection problem. . 27
12 Selection of the closest feasible pair of internal quartic control points. 27
13 Evaluation of a piecewise Bézier curve on a normalized local grid. 28
14 Verification of endpoint and mean preservation for the cubic method. 29
15 Verification of mean and derivative constraints for the exact quartic method. 29
16 Diagnostic derivative change for the bounded quartic method. 30
17 Verification of lower and upper bound violations. 30
18 Evaluation of reference values on the dense comparison grid. . 31
19 Computation of approximation error metrics. 31
20 Storage of method results for comparison. 32
21 Computation of the control-polygon roughness. 32
22 Graphical comparison of the interpolation methods. 33
23 Reading original data from the Excel worksheet. 35
24 Construction of support point indices in VBA. 36
25 Computation of empirical interval means in VBA. 36
26 Feasibility check for the bounded quartic construction in VBA. 37
27 Calling the bounded quartic Bézier construction in VBA. . . . 37
28 Evaluation of the piecewise quartic Bézier curve at a worksheet timestamp. 38